

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 4.2 实验名称 高速缓存 cache 设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 “-” 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支 (具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- 设计步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- 开发步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- 调试步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图,可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF,再插入到 L^AT_EX 中)

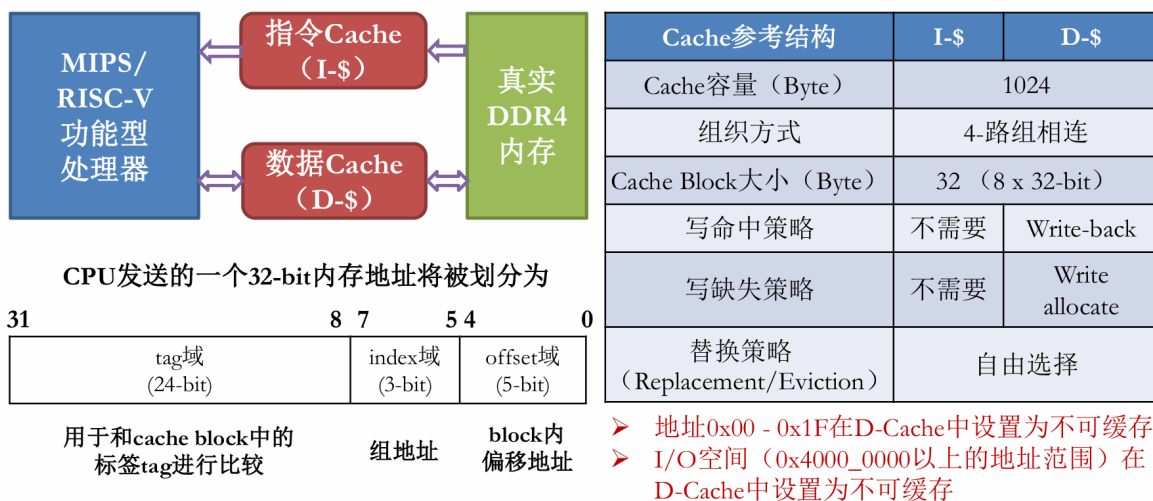


图 1: Cache 参考结构

Cache 和处理器及内存接口的数据位宽均为 32-bit (4-Byte)

1 iCache

iCache 的状态转移图如下图所示:

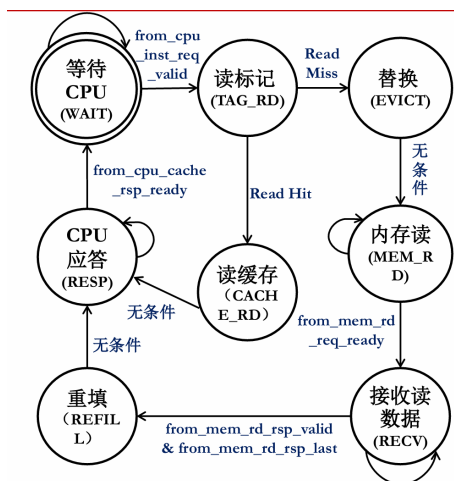


图 2: iCache 状态转移图

- WAIT:拉高 to_cpu_inst_req_ready 信号
- TAG_RD: 根据输入地址 index 域, 读出 4 路 valid+tag, 并与输入地址的 tag 域比较, 产生 Read Hit/Read miss 信号
- CACHE_RD:从命中的 cache block 中读出 32 byte,并根据输入地址的 offset 域,选择出要返回的指令码
- RESP:拉高 to_cpu_cache_rsp_valid 端口信号,并将要返回的指令码输出到 to_cpu_cache_rsp_data 端口
- EVICT:根据替换算法,从 4 路 cache block 中选择一个进行替换,将被替换 block 对应的 valid 清 0
- MEM_RD:向内存发送输入请求所在的 cache block 地址(32 byte 对齐地址),拉高 to_mem_rd_req_valid
- RECV:拉高 to_mem_rd_rsp_ready。每当 from_mem_rd_rsp_valid 拉高时,接收 4-byte from_mem_rd_rsp_data 直至 from_mem_rd_rsp_last 标记的最后一个 4-byte 数据已接收
- REFILL:将已收到的 32-byte 数据填入选中 cache block,同时更新对应的 tag 和 valid,并根据输入地址 offset 域,返回指令码

2 dCache

dCache 的状态转移图如下图所示:

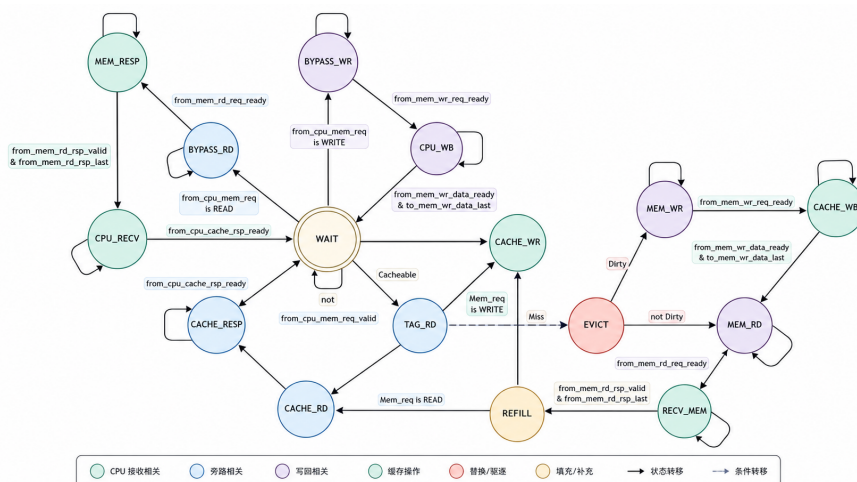


图 3: dCache 状态转移图

- WAIT:缓冲 CPU 访存的请求地址、读写类型、数据与选通信号,等待 CPU 访存请求
- TAG_RD:查询当前地址对应的标记位,判断目标数据块是否已命中缓存
- EVICT:基于 LRU 算法选取待替换缓存块,同时判断该块是否需要执行写回内存操作
- MEM_WR:向内存发起写请求
- CACHE_WB:采用 Burst 突发传输模式,将待替换缓存块的数据写回内存
- MEM_RD:向内存发送读请求
- RECV_MEM:采用 Burst 突发传输模式,从内存接收新缓存块的全部数据
- REFILL:将接收完成的数据写入对应缓存块,重置 LRU 计数器,并更新有效位、脏位与标记位
- CACHE_WR:将 CPU 下发的修改数据写入对应缓存块
- CACHE_RD:从命中的缓存块中读取所需数据
- CACHE_RESP:将缓存读出的数据返回至 CPU,同步更新 LRU 计数器
- BYPASS_RD:旁路读模式下,直接向内存发送读请求
- MEM_RESP:接收内存返回的读数据
- CPU_RECV:将内存传回的数据发送给 CPU
- BYPASS_WR:旁路写模式下,直接向内存发送写请求
- CPU_WB:将前期缓冲的 CPU 数据发送至内存完成写回

二、 开发成果(对代码开发流程的总结。需附上包含注释的 VerilogHDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 iCache 设计

请求地址字段划分及信号生成如下

```
assign {tag, index, offset} = from_cpu_inst_req_addr;
assign to_cpu_inst_req_ready = current_state[isWAIT];
assign to_cpu_cache_rsp_valid = current_state[isRESP];
assign to_mem_rd_req_valid = current_state[isMEM_RD];
assign to_mem_rd_req_addr = {tag, index, 5'b000000};
assign to_mem_rd_rsp_ready = current_state[isRECV] || rst;
```

tag 和 data 数组的创建及 ReadHit 与 ReadMiss 的判定与处理如下

```
genvar i_way;
generate
  for (i_way=0;i_way<`CACHE_WAY;i_way=i_way+1) begin : ways
    tag_array tags(
      .clk (clk),
      .waddr (index),
      .raddr (index),
      .wen (wen[i_way]),
      .wdata (tag),
      .rdata (tag_rdata[i_way])
    );
    assign tag_hit[i_way] = valid_array[i_way][index] && (tag_rdata[i_way] == tag);
    data_array datas(
      .clk (clk),
      .waddr (index),
```

```

        .raddr (index),
        .wen (wen[i_way]),
        .wdata ({
            data_recv[7],data_recv[6],data_recv[5],data_recv[4],
            data_recv[3],data_recv[2],data_recv[1],data_recv[0]
        }),
        .rdata (data_rdata[i_way])
    );
end
endgenerate

```

tag_hit 线路用来表示各 way 中 index 地址下的 tag 值是否与输入地址的 tag 域匹配。若四个 way 中存在一个 way 匹配,则 ReadHit,反之 ReadMiss。

```

assign ReadHit = tag_hit[0] | tag_hit[1] | tag_hit[2] | tag_hit[3];
assign ReadMiss = ~ReadHit;

assign way_selected = {(2){tag_hit[0]}} & 2'b00 |
    {(2){tag_hit[1]}} & 2'b01 |
    {(2){tag_hit[2]}} & 2'b10 |
    {(2){tag_hit[3]}} & 2'b11 ;
assign data_selected = data_rdata[way_selected];

```

当 ReadHit 时,way_selected 用来表示 tag 匹配的那一个 way。data_selected 用来表示此时输入地址对应的 cache 中缓存的数据值。为了生成向 cpu 回复的数据,还需要将 data_selected 偏移一段,并截取低位数据。

```

assign to_cpu_cache_rsp_data = data_selected >> {offset[4:0], 3'b000};

```

LRU_cnt 计数器如下:

```

integer i;
always @(posedge clk) begin
    if (rst) begin
        for (i=0;i<8;i=i+1) begin
            LRU_cnt[0][i] <= 0;
            LRU_cnt[1][i] <= 0;
            LRU_cnt[2][i] <= 0;
            LRU_cnt[3][i] <= 0;
        end
    end else if (!rst && current_state[isRESP] && next_state[isWAIT]) begin
        if (way_selected!=2'b00) LRU_cnt[0][index] <= LRU_cnt[0][index] + 1;
        if (way_selected!=2'b01) LRU_cnt[1][index] <= LRU_cnt[1][index] + 1;
        if (way_selected!=2'b10) LRU_cnt[2][index] <= LRU_cnt[2][index] + 1;
        if (way_selected!=2'b11) LRU_cnt[3][index] <= LRU_cnt[3][index] + 1;
    end else if (!rst && current_state[isREFILL]) begin
        LRU_cnt[way_LRU[index]][index] <= 4'b0000;
    end
end
end

```

我采用了 LRU 替换算法,基本思想是创建一个额外的计数器。当数据块从内存读入时 cache 时,将其置为

零。在接下来的每次访问时,没访问到的另外三路 cache 对应 index 下的计数器加一。替换时,选取当前计数器值最大的 cache 块替换即可。

```
genvar i_set;
generate
  for (i_set=0;i_set<`CACHE_SET;i_set=i_set+1) begin : sets
    assign way_LRU[i_set] =
      {(2){~valid_array[0][i_set]}} & 2'b00 |
      {(2){ valid_array[0][i_set] & ~valid_array[1][i_set]}} & 2'b01 |
      {(2){ valid_array[0][i_set] & valid_array[1][i_set] & ~valid_array[2][i_set]}} & 2'b10 |
      {(2){ valid_array[0][i_set] & valid_array[1][i_set] & valid_array[2][i_set] &
        ~valid_array[3][i_set]}} & 2'b11 |
      {(2){ valid_array[0][i_set] & valid_array[1][i_set] & valid_array[2][i_set] &
        valid_array[3][i_set]}} & (
        LRU_cnt[0][i_set] >= LRU_cnt[1][i_set] ?
        LRU_cnt[0][i_set] >= LRU_cnt[2][i_set] ?
        LRU_cnt[0][i_set] >= LRU_cnt[3][i_set] ? 2'b00 : 2'b11 :
        LRU_cnt[2][i_set] >= LRU_cnt[3][i_set] ? 2'b10 : 2'b11 :
        LRU_cnt[1][i_set] >= LRU_cnt[2][i_set] ?
        LRU_cnt[1][i_set] >= LRU_cnt[3][i_set] ? 2'b01 : 2'b11 :
        LRU_cnt[2][i_set] >= LRU_cnt[3][i_set] ? 2'b10 : 2'b11
      );
  end
endgenerate
```

valid 数组的更新逻辑以及对内存数据的接受逻辑如下

```
integer j;
always @(posedge clk) begin
  if (rst) begin
    for (j=0;j<8;j=j+1) begin
      valid_array[0][j] <= 0;
      valid_array[1][j] <= 0;
      valid_array[2][j] <= 0;
      valid_array[3][j] <= 0;
    end
  end else if (!rst && current_state[isEVICT]) begin
    valid_array[way_LRU[index]][index] <= 1'b0;
  end else if (!rst && current_state[isMEM_RD]) begin
    len <= 3'b000;
  end else if (!rst && current_state[isRECV] && from_mem_rd_rsp_valid) begin
    data_recv[len] <= from_mem_rd_rsp_data;
    len <= len + 1;
  end else if (!rst && current_state[isREFILL]) begin
    valid_array[way_LRU[index]][index] <= 1'b1;
  end
end

assign wen[0] = current_state[isREFILL] && way_LRU[index] == 2'b00;
assign wen[1] = current_state[isREFILL] && way_LRU[index] == 2'b01;
```

```

assign wen[2] = current_state[isREFILL] && way_LRU[index] == 2'b10;
assign wen[3] = current_state[isREFILL] && way_LRU[index] == 2'b11;

```

2 dCache 设计

在信号生成方面采用数据选择逻辑

```

assign to_cpu_mem_req_ready = current_state[isWAIT];
assign to_cpu_cache_rsp_valid = current_state[isCACHE_RESP] || current_state[isCPU_RECV];
assign to_cpu_cache_rsp_data = {(32){current_state[isCACHE_RESP]}} & data_selected >>
    {offset[4:0], 3'b000}
    | {(32){current_state[isCPU_RECV]}} & data_bypass;
assign to_mem_rd_req_valid = current_state[isMEM_RD] || current_state[isBYPASS_RD];
assign to_mem_rd_req_addr = {(32){current_state[isMEM_RD]}} & {tag, index, 5'b00000}
    | {(32){current_state[isBYPASS_RD]}} & Address;
assign to_mem_rd_req_len = {(8){current_state[isMEM_RD]}} & 7
    | {(8){current_state[isBYPASS_RD]}} & 0;
assign to_mem_rd_rsp_ready = current_state[isRECV_MEM] || current_state[isMEM_RESP] || rst;
assign to_mem_wr_req_valid = current_state[isMEM_WR] || current_state[isBYPASS_WR];
assign to_mem_wr_req_addr = {(32){current_state[isMEM_WR]}} & {tag_rdata[way_LRU[index]], index,
    5'b00000}
    | {(32){current_state[isBYPASS_WR]}} & Address;
assign to_mem_wr_req_len = {(8){current_state[isMEM_WR]}} & 7
    | {(8){current_state[isBYPASS_WR]}} & 0;
assign to_mem_wr_data_valid = current_state[isCACHE_WB] || current_state[isCPU_WB];
assign to_mem_wr_data = {(32){current_state[isCACHE_WB]}} & data_wb
    | {(32){current_state[isCPU_WB]}} & Write_data;
assign to_mem_wr_data_strb = {(4){current_state[isCACHE_WB]}}
    | {(4){current_state[isCPU_WB]}} & Write_strb;
assign to_mem_wr_data_last = current_state[isCACHE_WB] && len == 3'b000
    || current_state[isCPU_WB];

assign {tag, index, offset} = Address;

assign Cacheable = (!from_cpu_mem_req_addr[31:5]) && !from_cpu_mem_req_addr[31] &&
    !from_cpu_mem_req_addr[30];
assign Miss = !(tag_hit[0] || tag_hit[1] || tag_hit[2] || tag_hit[3]);
assign Dirty = valid_array[way_LRU[index]][index] && dirty_array[way_LRU[index]][index];

```

其中 Cacheable 信号通过判断输入地址是否落在特定范围实现。Miss 信号和前文一样,通过判断 tag_hit 线路实现。Dirty 信号通过读取 valid 数组和 dirty 数组实现。值得注意的是,只有当前 cache 块是有效时,才需要写回,因此 dirty 才有效。

wdata 和 wen 连线如下:

```

data_array datas(
    .clk (clk),
    .waddr (index),
    .raddr (index),
    .wen (wen[i_way]),

```

```

.wdata ({(256){current_state[isREFILL]}} & {
    data_recv[7],data_recv[6],data_recv[5],data_recv[4],
    data_recv[3],data_recv[2],data_recv[1],data_recv[0]
} | {(256){current_state[isCACHE_WR]}} & (
    data_rdata[i_way] & ~(
        224'b0,
        {(8){Write_strb[3]}},{(8){Write_strb[2]}},{(8){Write_strb[1]}},{(8){Write_strb[0]}}
    }<<{offset,3'b000}) | ({
        224'b0,
        Write_data &
        {(8){Write_strb[3]}},{(8){Write_strb[2]}},{(8){Write_strb[1]}},{(8){Write_strb[0]}}
    }<<{offset,3'b000})
)),
.rdata (data_rdata[i_way])
);

```

相较 icache, dcache 除了从内存能写数据到 cache 外, cpu 也能通过 write_data, write_strb 修改 cache 中的值。因此对于 data 寄存器的 wdata 和 wen 接口, 在接入前也需要对数据进行选择。

```

assign wen[0] = current_state[isREFILL] && way_LRU[index] == 2'b00 ||
    current_state[isCACHE_WR] && way_selected == 2'b00;
assign wen[1] = current_state[isREFILL] && way_LRU[index] == 2'b01 ||
    current_state[isCACHE_WR] && way_selected == 2'b01;
assign wen[2] = current_state[isREFILL] && way_LRU[index] == 2'b10 ||
    current_state[isCACHE_WR] && way_selected == 2'b10;
assign wen[3] = current_state[isREFILL] && way_LRU[index] == 2'b11 ||
    current_state[isCACHE_WR] && way_selected == 2'b11;

```

请求数据缓冲: 注意到 CPU 在成功发送请求后并不会等待 cache 完成一系列之后的操作, 因此需要对请求数据进行缓冲。

```

integer j;
always @(posedge clk) begin
    if (rst) begin
        for (j=0; j<8; j=j+1) begin
            valid_array[0][j] <= 0;
            valid_array[1][j] <= 0;
            valid_array[2][j] <= 0;
            valid_array[3][j] <= 0;
            LRU_cnt[0][j] <= 0;
            LRU_cnt[1][j] <= 0;
            LRU_cnt[2][j] <= 0;
            LRU_cnt[3][j] <= 0;
        end
    end else if (!rst && current_state[isEVICT]) begin
        valid_array[way_LRU[index]][index] <= 1'b0; //keep zero till finish clean
    //end else if (next_state[isMEM_WR]) begin //for mutually excluded if
        len <= 3'b000; //initialize the counter `len`
    end else if (!rst && !current_state[isCACHE_WB] && next_state[isCACHE_WB]) begin
        data_wb <= data_rdata[way_LRU[index]] >> {len, 5'b000000};
    end
end

```

```

len <= len + 1;
end else if (!rst && current_state[isCACHE_WB] && from_mem_wr_data_ready) begin
    data_wb <= data_rdata[way_LRU[index]] >> {len, 5'b000000};
    len <= len + 1;
end else if (!rst && current_state[isMEM_RD]) begin
    len <= 3'b000;
end else if (!rst && current_state[isRECV_MEM] && from_mem_rd_rsp_valid) begin
    data_recv[len] <= from_mem_rd_rsp_data;
    len <= len + 1;
end else if (!rst && current_state[isREFILL]) begin
    LRU_cnt[way_LRU[index]][index] <= 4'b0000;
    valid_array[way_LRU[index]][index] <= 1'b1; //finish clean
    dirty_array[way_LRU[index]][index] <= 1'b0;
end else if (!rst && current_state[isCACHE_RESP] && next_state[isWAIT]) begin
    if (way_selected!=2'b00) LRU_cnt[0][index] <= LRU_cnt[0][index] + 1;
    if (way_selected!=2'b01) LRU_cnt[1][index] <= LRU_cnt[1][index] + 1;
    if (way_selected!=2'b10) LRU_cnt[2][index] <= LRU_cnt[2][index] + 1;
    if (way_selected!=2'b11) LRU_cnt[3][index] <= LRU_cnt[3][index] + 1;
end else if (!rst && current_state[isCACHE_WR]) begin
    dirty_array[way_selected][index] <= 1'b1;
end else if (!rst && current_state[isMEM_RESP] && from_mem_rd_rsp_valid) begin
    data_bypass <= from_mem_rd_rsp_data;
end
end
end

```

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

1.1 cache 寄存器未正常写入

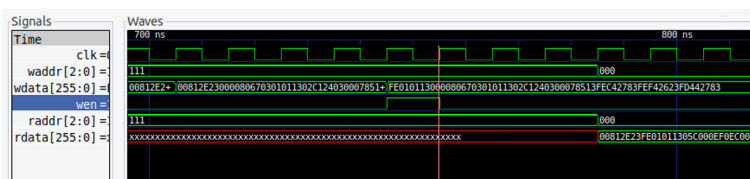


图 4: Cache 未正常写入波形图

如图,这里写地址 waddr 为 111,写入数据 wdata 有效,wen 为高电平,理应在红线所示时刻触发,将数据写入。但是,通过观察读地址 raddr 对应的数据 rdata 可知,该数据并没有正常写入。

出错原因是老师之前提供的 data_array.v 和 tag_array.v 中在定义 array 数组的长度时,使用了

```
reg [`DARRAY_DATA_WIDTH-1:0] array[ 1 << `DARRAY_ADDR_WIDTH - 1 : 0];
```

所示的运算。但事实上,因为左移的优先级低于减法,这个操作会先执行减一运算,再执行左移运算。导致数组大小的定义出了问题。

1.2 状态下标定义出错

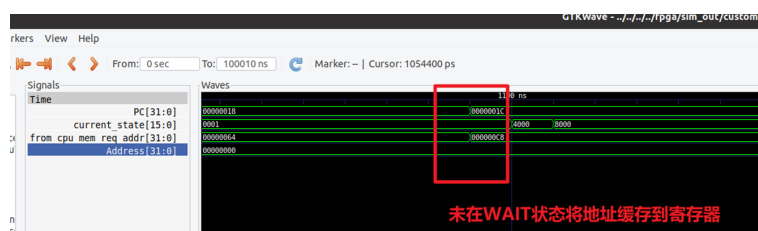


图 5: 缓冲失败波形图

如图,这里的 `from_cpu_mem_req_addr` 应该在 WAIT 状态结束时,缓冲到 Address 寄存器。但是波形显示并未成功缓冲。导致这个问题的原因在于状态下标定义出了失误,应该从 0 开始,我弄成了 1 开始

1.3 `to_mem_wr_data_last` 信号未正常拉高



图 6: 信号拉高失败波形图

在生成该信号时,之前错误使用了 `|` 符号而非 `||` 符号。虽然对于 1bit 数据两者功能类似,但是在这里仍然只能使用 `||` 符号。这是因为,该符号前的语句为 `len==3'b000`。如果使用 `|` 符号,由于 `|` 符号优先级高于 `==` 符号,代码将被解释为 `current_state[isCACHE_WB]len==(3'b000|current_state[isCPU_WB])`,与期望功能不符。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	cache 寄存器未正常写入	array	运算优先级	运算优先级	添加括号
2	状态下标定义出错	状态定义	缓冲失败	下标错误	修改下标
3	信号	生成信号	信号拉高失败	错误地使用逻辑符号	修正
3	配置文件	配置文件	忘记把 mips 改成 risc	太愚蠢了	修正

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

这次实验的 icache 部分原理简单,由于资料比较完善,易于实现。dcache 部分相较 icache 部分复杂一些,但是由于之前实现过 icache,加上 dcache 部分自由空间较大,因此相关设计和实现并不算非常复杂。另外由于 `data_array.v` 和 `tag_array.v` 如之前提到的那样存在一些错误,给波形调试增加了一些工作。所幸由于本地仿真工具较为便捷,该错误不难发现、纠正。

类别	名称	体会	后续计划
对 cpu 的理解	理解多周期 cpu	更加熟悉 cpu 运行原理,即几个阶段的整合	多复习讲义
代码编写	编写 verilog 并实现 always 逻辑	代码编写更加熟练	编写代码
debug 能力	寻找错误	寻找代码错误原因更加熟练	辅助工具实现 debug

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- 如何实现大于 4 byte 的指令/数据交互？

使用突发(Burst)传输。在发起请求后,连续传输多个数据。用 last 信号标志本次突发传输最后一个有效数据。用 len 字段标记本次突发读/写的长度。

- 如果用上述接口只传输 32-bit 数据(len = 0),last 在哪个位置拉高？

在第 0 个数据 valid 时,拉高 last 信号。

五、 实验时长

在课后,你花费了大约 18 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	3	认真学习课程内容,理解所需实现的板块
开发	5	完成实验代码的编写,包括多周期 cpu 文件,计数器等等
调试	10	进行测试样例的测试,不断修正错误

表 3: 实验时长